ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

SINH VIÊN THỰC HIỆN

Nguyễn Hoàng Hải Minh	24521066
Nguyễn Trần Kiều Phương	25521476

BÁO CÁO ĐỒ ÁN

NGHIÊN CỨU VÀ THIẾT KẾ

MÁY TÍNH BỎ TÚI THÔNG MINH

< Smart Pocket Calculator with STM32F103 >

MENTOR HƯỚNG DẪN: THANH KHỞI

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Qua quá trình nghiên cứu và thực hiện, đến nay nhóm chúng em đã hoàn thành đồ án của câu lạc bộ với đề tài: "Nghiên cứu và thiết kế máy tính bỏ túi thông minh sử dụng vi điều khiển STM32F103C8T6". Trong suốt quá trình thực hiện, nhóm đã gặp nhiều khó khăn và thử thách nhưng cũng nhận được rất nhiều sự hỗ trợ quý báu.

Do thời gian và năng lực có hạn, báo cáo không thể tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét từ các anh chị và các bạn để đồ án ngày càng được hoàn thiện hơn.

Trân trọng.

MỤC LỤC

Chương 1. Mục tiêu nhóm và giới hạn đồ án 5

Chương 2. Giới thiệu phần cứng và linh kiện sử dụng 7

Chương 3. Thiết kế hệ thống 12

Chương 4. Cài đặt phần mềm và giải thích code chi tiết 17

Chương 5. Tích hợp UART – Kết nối STM32 với máy tính 28

Chương 6. Kết quả thực thi và đánh giá 33

Tài liệu tham khảo 37

Chương 1. MỤC TIÊU CỦA NHÓM VÀ GIỚI HẠN CỦA ĐỒ ÁN

1.1. Thành viên nhóm và kế hoạch họp nhóm

STT	Họ và tên	MSSV	Chức vụ
1	Nguyễn Hoàng Hải Minh	24521066	Nhóm trưởng
2	Nguyễn Trần Kiều Phương	25521476	Thành viên

1.2. Mục tiêu hoạt động của nhóm

- Nghiên cứu và nắm vững kiến trúc ARM Cortex-M3, lập trình vi điều khiển STM32F103C8T6 bằng HAL Library trong môi trường STM32CubeIDE.
- Thiết kế và xây dựng máy tính bỏ túi thông minh tích hợp đồng hồ thời gian thực (RTC) sử dụng module DS3231.
- Hiển thị giao diện người dùng trực quan trên màn hình OLED SSD1306 128×64 pixel giao tiếp qua SPI.
- Tích hợp giao tiếp UART hai chiều: gửi log kết quả lên máy tính và nhận lệnh điều khiển từ PC.
- Rèn luyện kỹ năng làm việc nhóm, đọc tài liệu kỹ thuật tiếng Anh và viết báo cáo chuyên ngành.

1.3. Nội dung các công việc và phân công thực hiện

STT	Nội dung	Yêu cầu chi tiết	Người thực hiện
1	Thiết kế phần cứng	Vẽ sơ đồ kết nối, chọn linh kiện, lắp mạch thực tế.	Hải Minh
2	Lập trình máy tính & OLED	Code xử lý keypad, logic máy tính nâng cao (ưu tiên ×÷ trước ±), hiển thị OLED.	Kiều Phương
3	Lập trình RTC DS3231	Giao tiếp I2C, đọc/ghi thời gian, chỉnh giờ qua keypad và UART.	Cả nhóm
4	Tích hợp UART	Gửi log tự động, nhận và xử lý lệnh SET từ PC bằng interrupt.	Hải Minh
5	Viết báo cáo	Tổng hợp tài liệu, viết báo cáo, chuẩn bị slide thuyết trình.	Cả nhóm

1.4. Tính ứng dụng của đề tài

- Giúp sinh viên hiểu sâu về vi điều khiển 32-bit hiện đại (ARM Cortex-M3) và các giao tiếp ngoại vi phức tạp (SPI, I2C, UART).
- Sản phẩm tích hợp nhiều chức năng thiết thực: tính toán, hiển thị đồng hồ thời gian thực, giao tiếp với máy tính – có giá trị ứng dụng thực tế.
- Phương pháp thiết kế có thể mở rộng sang các ứng dụng nhúng khác như trạm đo lường, thiết bị IoT.

1.5. Giới hạn của nhóm với đồ án

- Đồ án chỉ hỗ trợ 4 phép tính cơ bản; chưa tích hợp phép tính nâng cao (lượng giác, mũ, căn).
- Màn hình OLED 128×64 giới hạn không gian hiển thị, chuỗi phép tính dài có thể bị cắt.

Chương 2. GIỚI THIỆU PHẦN CỨNG VÀ LINH KIỆN SỬ DỤNG

2.1. Vi điều khiển STM32F103C8T6 (Blue Pill)

Vi điều khiển STM32F103C8T6 là trung tâm xử lý của toàn bộ hệ thống, thuộc họ STM32F1 của hãng STMicroelectronics, sử dụng lõi xử lý ARM Cortex-M3 32-bit.

a) Lý do chọn STM32F103C8T6

- Hiệu năng mạnh: CPU 72 MHz, vượt trội so với Arduino UNO (16 MHz), đủ để xử lý đồng thời nhiều tác vụ.
- Tích hợp đầy đủ ngoại vi: SPI, I2C, UART, GPIO, Timer – đủ để điều khiển tất cả linh kiện trong đồ án.
- Hỗ trợ HAL Library và STM32CubeIDE: dễ dàng cấu hình ngoại vi bằng giao diện đồ họa, sinh code tự động.
- Giá thành rẻ, dễ mua tại thị trường Việt Nam.

b) Thông số kỹ thuật chính

Thông số	Giá trị
CPU	ARM Cortex-M3, 72 MHz
Bộ nhớ Flash	64 KB
RAM (SRAM)	20 KB
Số chân GPIO	37 chân I/O
Ngoại vi giao tiếp	3× USART, 2× SPI, 2× I2C, 1× USB
Điện áp hoạt động	2.0V – 3.6V
Nạp code	ST-Link (SWD) hoặc USB Bootloader

2.2. Màn hình OLED SSD1306 128×64 (SPI)

Màn hình OLED SSD1306 độ phân giải 128×64 pixel được giao tiếp với STM32 qua SPI1. So với LCD1602 thông thường, OLED có nhiều ưu điểm vượt trội:

- Độ phân giải cao (128×64 px): hiển thị đồng thời đồng hồ, biểu thức tính toán và kết quả rõ ràng.
- Tốc độ nhanh: SPI đạt tốc độ truyền cao hơn I2C, phù hợp cập nhật màn hình liên tục.

- Tự phát sáng: không cần đèn nền, tiêu thụ điện năng thấp.

Bảng kết nối chân SSD1306 với STM32:

Chân OLED	Tín hiệu	Chân STM32	Ghi chú
VCC	Nguồn	3.3V	Cấp nguồn
GND	Đất chung	GND	
D0 (CLK)	SPI Clock	PA5 (SPI1_SCK)	Xung nhịp SPI
D1 (MOSI)	SPI Data	PA7 (SPI1_MOSI)	Dữ liệu SPI
CS	Chip Select	PA4 (Oled_CS)	Chọn chip OLED
DC	Data/Command	PB1 (Oled_DC)	Chọn lệnh/dữ liệu
RES	Reset	PB0 (Oled_RES)	Reset OLED

2.3. Module thời gian thực DS3231

DS3231 là module RTC (Real-Time Clock) độ chính xác cao, giao tiếp I2C với địa chỉ 0xD0. Module có tích hợp bộ dao động tinh thể và cảm biến nhiệt độ on-chip, đảm bảo sai số rất nhỏ (< 2 ppm). DS3231 sử dụng pin CR2032 dự phòng để giữ thời gian khi mất điện.

Bảng kết nối DS3231 với STM32:

Chân DS3231	Tín hiệu	Chân STM32	Ghi chú
VCC	Nguồn	3.3V	
GND	Đất chung	GND	
SCL	I2C Clock	PB6 (I2C1_SCL)	Xung nhịp I2C
SDA	I2C Data	PB7 (I2C1_SDA)	Dữ liệu I2C

2.4. Bàn phím ma trận 4×4

Bàn phím ma trận 4×4 gồm 16 phím được tổ chức thành 4 hàng và 4 cột, sử dụng 8 chân GPIO. Kỹ thuật quét hàng-cột được áp dụng để nhận biết phím được nhấn.

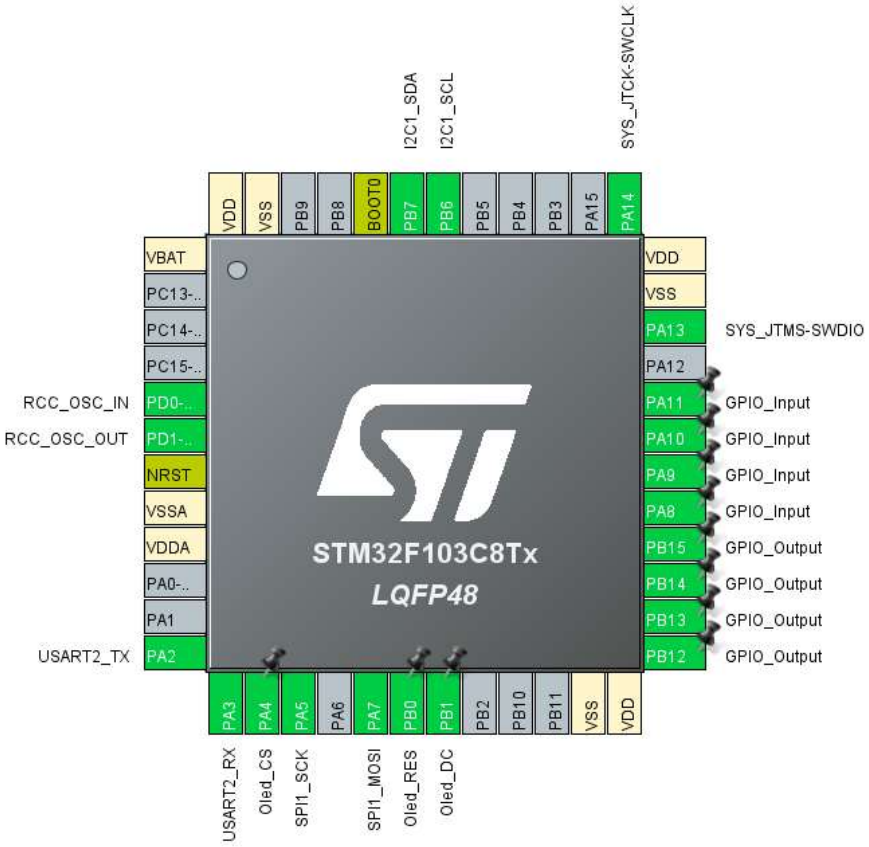
Bảng đồ phím (keypad_map):

	Cột 1 (PA8)	Cột 2 (PA9)	Cột 3 (PA10)	Cột 4 (PA11)
Hàng 1 (PB12)	1	2	3	A (+)
Hàng 2 (PB13)	4	5	6	B (−)
Hàng 3 (PB14)	7	8	9	C (×)
Hàng 4 (PB15)	* (xoá)	0	# (=)	D (/)

2.5. Module USB-to-UART (CP2102 / CH340)

Module USB-TTL chuyển đổi giao tiếp UART của STM32 thành USB để kết nối với máy tính. STM32 sử dụng USART2 trên chân PA2 (TX) và PA3 (RX) với tốc độ Baud Rate 115200 bps.

2.6. Cấu hình các chân



Chương 3. THIẾT KẾ HỆ THỐNG

3.1. Sơ đồ khối tổng thể

Khối	Thành phần	Chức năng
Trung tâm điều khiển	STM32F103C8T6	Xử lý toàn bộ logic hệ thống
Nhập liệu	Keypad 4×4	Nhận giá trị số và lệnh từ người dùng
Hiển thị	OLED SSD1306 128×64	Hiển thị giao diện, kết quả, đồng hồ
Thời gian thực	DS3231 RTC	Lưu trữ và cung cấp giờ/ngày chính xác
Giao tiếp máy tính	USART2 + USB-TTL	Log dữ liệu và nhận lệnh từ PC

3.2. Các giao tiếp sử dụng trong hệ thống

- SPI1: STM32 (Master) ↔ OLED SSD1306 (Slave) – tốc độ cao, cập nhật màn hình liên tục.
- I2C1 (100 kHz): STM32 (Master) ↔ DS3231 (Slave) – đọc/ghi thời gian thực.
- GPIO Output (PB12–PB15): STM32 → 4 hàng Keypad – kỹ thuật quét hàng.
- GPIO Input Pull-Up (PA8–PA11): 4 cột Keypad → STM32 – đọc trạng thái phím.
- USART2 (115200 bps): STM32 ↔ USB-TTL ↔ Máy tính – giao tiếp hai chiều.

3.3. Các chế độ hoạt động

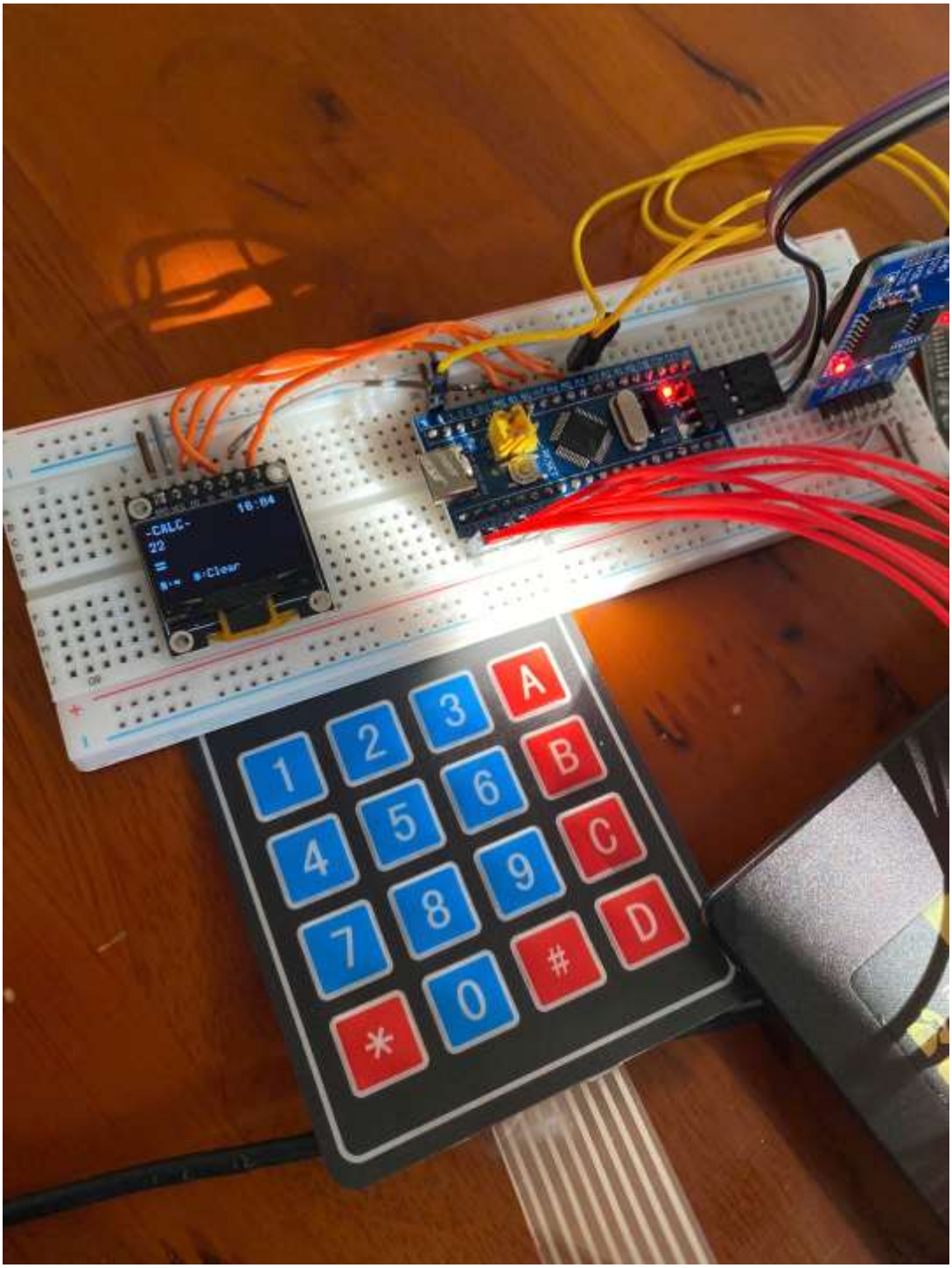
Mode 0: Máy tính + Đồng hồ (chế độ mặc định)

Sau khi khởi động, hệ thống vào Mode 0. Màn hình OLED hiển thị 4 vùng thông tin đồng thời:

- Góc trái trên: Nhãn '-CALC-' cho biết đang ở chế độ máy tính.
- Góc phải trên: Đồng hồ ngắn (HH:MM) lấy từ DS3231, cập nhật mỗi vòng lặp.
- Vùng giữa: Hiển thị chuỗi phép tính (VD: '12 + 3 × 5') và kết quả '= 27'.
- Dòng dưới: Gợi ý phím '#:= *:Clear'.
- Nếu không bấm phím quá 15 giây: tự động chuyển sang hiển thị đồng hồ toàn màn hình.

Mode 1: Chỉnh giờ (SET TIME)

Chuyển sang Mode 1 bằng cách GIỮ phím 'A' hơn 2 giây. Màn hình hướng dẫn nhập 4 chữ số HHMM. Nhấn '#' để lưu (sau khi kiểm tra hợp lệ h < 24, m < 60), nhấn '*' để xoá.



Chương 4. CÀI ĐẶT PHẦN MỀM VÀ GIẢI THÍCH CODE CHI TIẾT

4.1. Cấu trúc dự án trong STM32CubeIDE

- main.c: File chương trình chính, chứa toàn bộ logic ứng dụng.
- ssd1306.c / ssd1306.h: Thư viện điều khiển màn hình OLED SSD1306.
- ssd1306_fonts.c / ssd1306_fonts.h: Dữ liệu font chữ (6×8, 7×10, 11×18, 16×26).
- ssd1306_conf.h: File cấu hình thư viện (SPI, chân CS/DC/RES).
- stm32f1xx_hal_msp.c: Cấu hình cấp thấp cho SPI1, I2C1, USART2.

4.2. Cấu hình ngoại vi (STM32CubeMX)

Ngoại vi	Chức năng	Thông số	Chân sử dụng
SPI1	OLED SSD1306	Master, CPOL=0, CPHA=1, /8	PA5(SCK), PA7(MOSI), PA4(CS), PB1(DC), PB0(RST)
I2C1	DS3231 RTC	Standard 100 kHz	PB6(SCL), PB7(SDA)
USART2	Giao tiếp PC	115200 bps, 8N1	PA2(TX), PA3(RX)
GPIO Output	Hàng Keypad	Push-Pull, No Pull	PB12, PB13, PB14, PB15
GPIO Input	Cột Keypad	Pull-Up	PA8, PA9, PA10, PA11
Clock	Hệ thống	HSE 8MHz × 9 = 72 MHz	PLL, Flash Latency = 2

4.3. Giải thích các hàm cơ bản

a) Hàm quét bàn phím: Keypad_Scan()

Hàm thực hiện kỹ thuật quét hàng-cột (row scanning). Nguyên lý hoạt động:

- Đặt tất cả 4 hàng (PB12–PB15) lên mức HIGH.
- Lần lượt kéo từng hàng xuống mức LOW.
- Đọc 4 chân cột (PA8–PA11): nếu có chân nào xuống 0 → phím tương ứng đang được nhấn.
- Trả về ký tự tương ứng từ mảng keypad_map[4][4] hoặc 0 nếu không có phím nào.

b) Hàm xử lý phím đặc biệt: Button_Handle()

Xử lý phím '+' theo thời gian nhấn – phân biệt nhấn nhanh và giữ lâu:

```
uint8_t Button_Handle(char key) {
    // Phím '+' được nhấn
    if (key == '+') {
        // Lần đầu nhấn: lưu thời điểm bắt đầu nhấn
        if (isPress == 0) { pressTime = HAL_GetTick(); isPress = 1; }
        // Đã nhấn quá 2 giây → chuyển mode
        else if (isPress == 1 && (HAL_GetTick() - pressTime > 2000)) {
            mode = (mode == 0) ? 1 : 0;
            isPress = 2; return 2; // Báo hiệu đã đổi mode
        }
        return 0; // Đang giữ, chờ tiếp
    } else {
        // Nhả phím: nếu chưa đủ 2s → dùng như phím '+' bình thường
        if (isPress == 1) { isPress = 0; return 1; }
        isPress = 0; return 0;
    }
}
```

c) Hàm định dạng số: format_num()

Hàm này giải quyết vấn đề hiển thị số thực: nếu là số nguyên thì không in '.00', nếu có phần thập phân thì in 2 chữ số sau dấu phẩy. Đặc biệt xử lý trường hợp số âm gần 0 (VD: -0.50):

```
void format_num(char* buf, float val) {
    int int_part = (int)val;
    // Nếu là số nguyên: in không có phần thập phân
    if (val == int_part) sprintf(buf, "%d", int_part);
    else {
        int dec_part = (int)(fabs(val - int_part) * 100);
        // Xử lý trường hợp -0.50 (int_part = 0 nhưng val âm)
        if (val < 0 && int_part == 0)
            sprintf(buf, "-0.%02d", dec_part);
        else
            sprintf(buf, "%d.%02d", int_part, dec_part);
    }
}
```

4.4. Logic máy tính nâng cao – Ưu tiên × ÷ trước ± (Smart Calculator)

Đây là một trong những tính năng nổi bật nhất của đồ án. Thay vì tính từ trái sang phải đơn giản, hệ thống triển khai đúng quy tắc toán học: nhân và chia được thực hiện trước cộng và trừ.

a) Cấu trúc dữ liệu lưu trữ chuỗi phép tính

Thay vì chỉ lưu 2 số (num1, num2), hệ thống sử dụng mảng để lưu toàn bộ chuỗi phép tính:

```
float nums[10];     // Lưu tối đa 10 số trong 1 phép tính
char  ops[10];      // Lưu tối đa 10 toán tử tương ứng
int   step_idx;     // Con trỏ chỉ đang ở số thứ mấy
uint8_t is_typing_number; // Cờ đang gõ dở một số
```

Ví dụ: Phép tính '2 + 3 × 5 − 1' sẽ được lưu như sau:

Mảng	Index 0	Index 1	Index 2	Index 3	Index 4	...
nums[]	2	3	5	1	—	...
ops[]	+	×	−	—	—	...

b) Thuật toán Smart_Calculator() – 2 lượt duyệt

Thuật toán thực hiện theo 2 bước (giống cách tính tay):

- Bước 1 – Lượt 1: Duyệt mảng ops[], xử lý tất cả phép × và ÷ trước. Kết quả được gộp ngay vào phần tử đứng trước trong mảng tạm temp_n[].
- Bước 2 – Lượt 2: Duyệt mảng tạm, xử lý các phép + và − còn lại theo thứ tự từ trái sang phải.

Code chi tiết của thuật toán:

```
float Smart_Calculator() {
    float temp_n[10];
    char  temp_o[10];
    int   t_idx = 0;
```

```
    temp_n[0] = nums[0]; // Khởi tạo số đầu tiên
```

```
    // ─── LƯỢT 1: Xử lý nhân (×) và chia (÷) trước ────────────────
    for (int i = 0; i < step_idx; i++) {
        if (ops[i] == 'x') {
            // Gộp kết quả nhân vào phần tử hiện tại
            temp_n[t_idx] = temp_n[t_idx] * nums[i+1];
        }
        else if (ops[i] == '/') {
            // Kiểm tra chia cho 0 trước khi thực hiện
            if (nums[i+1] == 0) return INFINITY; // Báo lỗi chia cho 0
            temp_n[t_idx] = temp_n[t_idx] / nums[i+1];
        }
        else {
            // Gặp + hoặc -: lưu lại toán tử, chuyển sang phần tử mới
            temp_o[t_idx] = ops[i];
```

```
            t_idx++;
            temp_n[t_idx] = nums[i+1];
        }
    }
```

```
    // ─── LƯỢT 2: Xử lý cộng (+) và trừ (-) còn lại ──────────────
    float final_result = temp_n[0];
    for (int i = 0; i < t_idx; i++) {
        if (temp_o[i] == '+') final_result += temp_n[i+1];
        if (temp_o[i] == '-') final_result -= temp_n[i+1];
    }
    return final_result;
}
```

Ví dụ minh họa với phép tính '2 + 3 × 5 − 1':

- Sau Lượt 1: temp_n = [2, 15, 1], temp_o = ['+', '−'] (số 3 và 5 đã được nhân thành 15).
- Sau Lượt 2: final_result = 2 + 15 − 1 = 16 ✓ (đúng theo quy tắc toán học).
- Nếu tính từ trái sang phải (sai): 2+3=5, 5×5=25, 25−1=24 ✗.

c) Hàm xây dựng chuỗi hiển thị: Build_Display_String()

Hàm này đọc mảng nums[] và ops[] để tạo ra chuỗi văn bản hiển thị lên OLED, ví dụ: '2 + 3 × 5':

```
void Build_Display_String() {
    char temp_str[15];
    calc_display_str[0] = '\0'; // Xoá chuỗi cũ
```

```
    for (int i = 0; i <= step_idx; i++) {
        // In số nếu đang gõ dở hoặc đã xong
        if (i < step_idx || is_typing_number) {
            format_num(temp_str, nums[i]);
            strcat(calc_display_str, temp_str);
        }
        // In toán tử giữa các số
        if (i < step_idx) {
            int len = strlen(calc_display_str);
            calc_display_str[len] = ' ';
            calc_display_str[len+1] = ops[i];
            calc_display_str[len+2] = ' ';
            calc_display_str[len+3] = '\0';
        }
    }
}
```

4.5. Xử lý lỗi toàn diện trong Calculator_HandleKey()

Đây là phần quan trọng giúp hệ thống hoạt động ổn định, ngăn chặn các tình huống nhập sai của người dùng. Có 6 loại lỗi được xử lý:

Lỗi 1: Chốt chặn bàn phím khi có lỗi

Khi biến error_msg[] chứa nội dung (có lỗi), toàn bộ bàn phím bị 'đóng băng' – hệ thống không nhận bất kỳ phím nào ngoại trừ '*' để xoá lỗi:

```
if (error_msg[0] != '\0') {
    // Chỉ nhận phím '*' để xoá lỗi và reset
    if (key == '*') {
        for(int i=0; i<10; i++) { nums[i] = 0; ops[i] = 0; }
        step_idx = 0; is_typing_number = 0;
        has_result = 0; error_msg[0] = '\0';
    }
    return; // Bỏ qua tất cả phím khác
}
```

Lỗi 2: Số nhập vào quá lớn (Too large!)

Trước khi cộng thêm chữ số mới vào nums[step_idx], hệ thống tính thử kết quả. Nếu vượt 9.999.999 thì báo lỗi ngay, không cho nhập thêm:

```
if (key >= '0' && key <= '9') {
    // Tính thử nếu thêm chữ số này vào
    float temp_val = nums[step_idx] * 10 + (key - '0');
    // Giới hạn 7 chữ số: tối đa 9.999.999
    if (temp_val > 9999999 || temp_val < -9999999) {
        sprintf(error_msg, "Too large!");
        UART_Print("[ERR] Input Too Large!\r\n");
        return;
    }
    nums[step_idx] = temp_val;
    is_typing_number = 1;
}
```

Lỗi 3: Không có số đầu tiên (No num1!)

Khi người dùng nhấn phím × hoặc / ngay từ đầu mà chưa nhập số nào, hệ thống báo lỗi. Riêng phím + và − được xử lý đặc biệt: cho phép nhập '+2' hay '−3' (hiểu ngầm là 0 + 2 hoặc 0 − 3):

```
// Bấm phép tính khi chưa có số nào (step_idx=0, chưa gõ số)
if (!is_typing_number && step_idx == 0) {
    if (key == 'x' || key == '/') {
        // × và ÷ không có nghĩa khi không có số trước
        sprintf(error_msg, "No num1!");
        UART_Print("[ERR] No num1!\r\n");
        return;
```

```
    } else if (key == '+' || key == '-') {
        // Cho phép nhập +2 hoặc -3 từ đầu: hiểu là 0 +/- số
        nums[0] = 0; // Số đầu ngầm định là 0
        ops[0] = key;
        step_idx = 1;
        return;
    }
}
```

Lỗi 4: Không có số thứ hai (No num2!)

Khi người dùng nhấn '#' (=) sau khi đã nhập phép tính nhưng chưa nhập số thứ hai (is_typing_number = 0 mà step_idx > 0):

```
else if (key == '#') {
    if (!is_typing_number && step_idx > 0) {
        // Đã nhập toán tử nhưng chưa có số theo sau
        sprintf(error_msg, "No num2!");
        UART_Print("[ERR] No num2!\r\n");
        return;
    }
```

Lỗi 5: Chia cho 0 (Div by 0!)

Được phát hiện bên trong hàm Smart_Calculator() khi gặp toán tử '/' mà số kế tiếp bằng 0. Hàm trả về INFINITY, và Calculator_HandleKey() kiểm tra kết quả trả về:

```
// Trong Smart_Calculator():
else if (ops[i] == '/') {
    if (nums[i+1] == 0) return INFINITY; // Báo hiệu lỗi chia 0
    temp_n[t_idx] = temp_n[t_idx] / nums[i+1];
}
```

```
// Trong Calculator_HandleKey() - sau khi gọi Smart_Calculator():
float result = Smart_Calculator();
if (isinf(result) || isnan(result)) {
    sprintf(error_msg, "Div by 0!");
    UART_Print("[ERR] Math Error: Div by 0!\r\n");
    return;
}
```

Lỗi 6: Kết quả tràn số (Overflow!)

Dù số nhập vào được giới hạn 7 chữ số, nhưng kết quả phép nhân vẫn có thể vượt giới hạn (VD: 9999999 × 9999999). Sau khi tính xong, kết quả được kiểm tra:

```
// Kiểm tra kết quả sau khi tính
if (result > 9999999 || result < -9999999) {
    sprintf(error_msg, "Overflow!");
    UART_Print("[ERR] Result Overflow!\r\n");
    return;
```

```
}
```

Bảng tổng hợp các loại lỗi và cách xử lý:

STT	Loại lỗi	Điều kiện phát sinh	Thông báo OLED	Log UART
1	Số nhập quá lớn	nums[i] × 10 + digit > 9.999.999	Too large!	[ERR] Input Too Large!
2	Thiếu số đầu	Nhấn × hoặc / khi chưa có số	No num1!	[ERR] No num1!
3	Thiếu số thứ hai	Nhấn # khi đã có toán tử nhưng chưa có số sau	No num2!	[ERR] No num2!
4	Chia cho 0	Toán tử / gặp số kế tiếp = 0	Div by 0!	[ERR] Math Error: Div by 0!
5	Kết quả tràn số	|result| > 9.999.999	Overflow!	[ERR] Result Overflow!
6	RTC mất kết nối	HAL_I2C_Mem_Read() != HAL_OK	RTC Error	(không gửi)

Chương 5. TÍCH HỢP UART – KẾT NỐI STM32 VỚI MÁY TÍNH

5.1. Tổng quan về chức năng UART trong đồ án

Hệ thống triển khai UART hai chiều hoàn chỉnh thông qua USART2:

- Chiều gửi (TX): STM32 chủ động gửi log kết quả máy tính và thông báo lỗi lên PC.
- Chiều nhận (RX): PC gửi lệnh 'SET HH:MM' xuống STM32 để cập nhật đồng hồ DS3231 từ xa.

5.2. Kết nối phần cứng

STM32F103 (Blue Pill)	Dây nối	Module USB-TTL
PA2 (USART2 TX)	——————→	RX
PA3 (USART2 RX)	←——————	TX
GND	——————	GND

Lưu ý: Chỉ cần 3 dây TX, RX và GND. Không nối 3.3V/5V nếu board STM32 đã có nguồn riêng.

5.3. Cơ chế nhận UART bằng Interrupt (không chặn vòng lặp)

Đây là điểm kỹ thuật quan trọng: thay vì dùng HAL_UART_Receive() (blocking – chặn cả hệ thống chờ dữ liệu), đồ án sử dụng HAL_UART_Receive_IT() (interrupt – không blocking). Nhờ vậy vòng lặp chính vẫn chạy bình thường, bàn phím và màn hình không bị trễ.

a) Khởi động lắng nghe UART trong main()

```
// Trong main(), sau khi khởi tạo UART:
HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
// → Yêu cầu MCU ngắt mỗi khi nhận được 1 byte
```

b) Hàm callback – xử lý từng byte nhận được

Mỗi khi nhận được 1 byte, hàm HAL_UART_RxCpltCallback() được gọi tự động bởi interrupt:

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART2) {
        // Gặp ký tự xuống dòng: lệnh hoàn chỉnh
        if (rx_byte == '\n' || rx_byte == '\r') {
```

```
            if (rx_index > 0) {
                rx_buf[rx_index] = '\0'; // Kết thúc chuỗi
                // Bật cờ báo hiệu: đã có lệnh sẵn sàng xử lý
                cmd_ready = 1;
            }
        } else {
            // Chưa xuống dòng: ghi thêm ký tự vào buffer
            if (rx_index < 30) {
                rx_buf[rx_index++] = rx_byte;
            }
        }
        // Quan trọng: kích hoạt lại interrupt để nhận byte tiếp theo
        HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
    }
}
```

c) Hàm 'cấp cứu' – tự khởi động lại khi bị lỗi

Trong một số trường hợp (nghẽn dữ liệu, nhiễu đường truyền), bộ nhận UART có thể bị treo. Hàm ErrorCallback() tự động phục hồi:

```
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart) {
    if (huart->Instance == USART2) {
        // Xoá cờ lỗi Overrun (phổ biến nhất)
        __HAL_UART_CLEAR_OREFLAG(huart);
        // Kích hoạt lại bộ nhận
        HAL_UART_Receive_IT(&huart2, &rx_byte, 1);
    }
}
```

5.4. Xử lý lệnh nhận được trong vòng lặp chính

Biến cmd_ready = 1 chỉ được kiểm tra trong vòng while(1), đảm bảo lệnh luôn được xử lý ở thread chính, không xử lý trong interrupt (tránh lỗi tương tranh):

```
if (cmd_ready == 1) {
    // Kiểm tra lệnh 'SET HH:MM' (VD: 'SET 14:30')
    if (strncmp((char*)rx_buf, "SET ", 4) == 0
        && strlen((char*)rx_buf) >= 9) {
        // Cắt lấy giờ (vị trí 4, 5) và phút (vị trí 7, 8)
        int h = (rx_buf[4]-'0')*10 + (rx_buf[5]-'0');
        int m = (rx_buf[7]-'0')*10 + (rx_buf[8]-'0');
```

```
        if (h >= 0 && h < 24 && m >= 0 && m < 60) {
            // Ghi thời gian mới vào DS3231
            Set_Time(0, m, h, 5, time.dayofmonth, time.month,
time.year);
            UART_Print("[SYS] Time Updated from PC!\r\n");
        } else {
            UART_Print("[ERR] Invalid Time format!\r\n");
        }
```

```
    } else {
        UART_Print("[ERR] Unknown Command. Try 'SET HH:MM'\r\n");
    }
    // Dọn dẹp buffer, sẵn sàng nhận lệnh tiếp theo
    rx_index = 0; cmd_ready = 0;
}
```

5.5. Các định dạng log gửi lên máy tính

Loại log	Định dạng	Ví dụ cụ thể
[MAY TINH]	[MAY TINH] <bieu_thuc> = <ket_qua>	[MAY TINH] 2 + 3 x 5 = 17.00
[ERR]	[ERR] <mo_ta_loi>	[ERR] Math Error: Div by 0!
[SYS]	[SYS] <thong_bao_he_thong>	[SYS] Time Updated from PC!

5.6. Hướng dẫn kiểm tra trên máy tính

Bước 1: Cài đặt driver USB-TTL

- CP2102: Tải driver tại silabs.com/developers/usb-to-uart-bridge-vcp-drivers.
- CH340: Tìm 'CH340 driver' và cài đặt. Kiểm tra Device Manager để xác nhận COM port.

Bước 2: Mở Serial Monitor

- PuTTY (Windows): Chọn 'Serial' → nhập COM port → Speed: 115200 → Open.
- Tera Term: Serial → chọn COM port → Setup → Serial Port → Baud: 115200.
- VS Code: Extension 'Serial Monitor' (Microsoft) – tích hợp ngay trong trình soạn thảo.

Bước 3: Gửi lệnh điều khiển

Gõ lệnh trong ô nhập của Serial Monitor, kết thúc bằng Enter (\n hoặc \r\n):

```
SET 14:30     → Đặt đồng hồ DS3231 về 14 giờ 30 phút
SET 08:05     → Đặt đồng hồ về 08 giờ 05 phút
SET 25:00     → [ERR] Invalid Time format! (giờ không hợp lệ)
```

Chương 6. KẾT QUẢ THỰC THI VÀ ĐÁNH GIÁ

6.1. Kết quả hoạt động của sản phẩm

Chức năng	Trạng thái	Ghi chú
Máy tính 4 phép tính cơ bản	✓ Đạt	Hỗ trợ số nguyên và thập phân, đến 7 chữ số
Ưu tiên × ÷ trước ± (thuật toán Smart)	✓ Đạt	Đúng theo quy tắc toán học
Nhập biểu thức dài nhiều bước	✓ Đạt	Tối đa 10 số trong một chuỗi phép tính
Xử lý 6 loại lỗi toàn diện	✓ Đạt	Thông báo rõ ràng, chỉ xoá bằng phím *
Đồng hồ thời gian thực (DS3231)	✓ Đạt	Duy trì khi mất điện nhờ pin CMOS
Hiển thị đồng hồ sau 15s không bấm	✓ Đạt	Chuyển chế độ tự động
Chỉnh giờ qua keypad	✓ Đạt	Định dạng HHMM, kiểm tra hợp lệ
Gửi log kết quả lên PC qua UART	✓ Đạt	Format [MAY TINH], [ERR], [SYS]
Nhận lệnh SET HH:MM từ PC	✓ Đạt	Interrupt-driven, không blocking
Tự phục hồi UART khi lỗi đường truyền	✓ Đạt	HAL_UART_ErrorCallback() tự reset

6.2. Đánh giá ưu điểm và hạn chế

Ưu điểm

- Thuật toán Smart_Calculator() xử lý đúng thứ tự ưu tiên phép tính, vượt trội so với các đồ án tương tự chỉ tính từ trái sang phải.
- Hệ thống xử lý lỗi toàn diện với 6 loại lỗi được phân loại rõ ràng, thông báo cụ thể cả trên OLED lẫn qua UART.
- UART hai chiều với interrupt: gửi log tự động và nhận lệnh từ PC mà không làm chậm hệ thống.
- Cơ chế 'cấp cứu' HAL_UART_ErrorCallback() đảm bảo kết nối UART bền vững, tự phục hồi khi lỗi.
- Giao diện người dùng trực quan: hiển thị toàn bộ biểu thức tính toán theo thời gian thực trên OLED.

Hạn chế và hướng cải tiến

- Sử dụng HAL_Delay() gây blocking trong vòng lặp – có thể cải tiến bằng FreeRTOS hoặc timer interrupt.
- Chưa hỗ trợ phép tính nâng cao (lượng giác, mũ, căn bậc hai).
- Chuỗi phép tính dài có thể bị cắt khi hiển thị trên OLED 128×64.
- Chức năng chỉnh ngày/tháng/năm qua UART chưa được tích hợp (chỉ chỉnh được giờ:phút).
- Có thể phát triển thêm giao diện Python/C# trên PC để hiển thị lịch sử tính toán.

6.3. Nhận xét kết quả học tập

Qua việc thực hiện đồ án này, nhóm đã thu được nhiều kết quả học tập thiết thực:

- Nắm vững kiến trúc ARM Cortex-M3 và cách sử dụng STM32 HAL Library trong dự án thực tế.
- Hiểu sâu về các giao tiếp ngoại vi SPI, I2C, UART và các cơ chế interrupt, callback trong lập trình nhúng.
- Rèn luyện kỹ năng thiết kế thuật toán máy tính với xử lý ưu tiên phép tính và xử lý lỗi toàn diện.
- Có kinh nghiệm debug phần cứng và phần mềm bằng UART – kỹ năng thiết yếu trong lập trình nhúng chuyên nghiệp.

TÀI LIỆU THAM KHẢO

- STMicroelectronics. (2015). STM32F103x8 Datasheet. Truy cập tại: https://www.st.com/resource/en/datasheet/stm32f103c8.pdf
- STMicroelectronics. (2021). RM0008 Reference Manual: STM32F1xx. Truy cập tại: https://www.st.com/resource/en/reference_manual/rm0008.pdf
- STMicroelectronics. (2023). STM32CubeIDE User Guide. Truy cập tại: https://www.st.com/en/development-tools/stm32cubeide.html
- Maxim Integrated. (2015). DS3231 Datasheet: Extremely Accurate I2C-Integrated RTC. Truy cập tại: https://datasheets.maximintegrated.com/en/ds/DS3231.pdf
- Solomon Systech. (2008). SSD1306 Datasheet: 128×64 Dot Matrix OLED Driver with Controller.
- Alekseev, A. (2018). stm32-ssd1306: SSD1306 OLED library for STM32. GitHub. https://github.com/afiskon/stm32-ssd1306